SDLC

Software Development Life Cycle

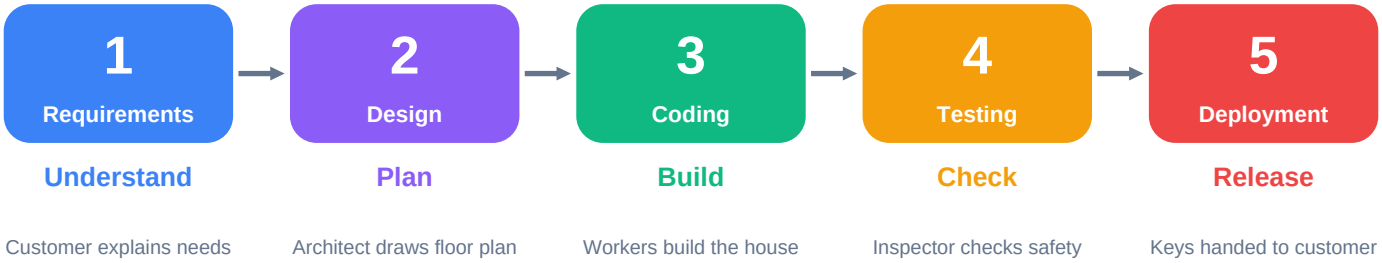
An easy, visual explanation with a real-world login example

Study guide for software testers

SDLC is the step-by-step process used to **develop software, test it, and deliver it** to customers.

Imagine a customer wants to build a house. You cannot immediately start laying bricks. First you understand what the customer wants, prepare a plan, construct the house, inspect it, and finally hand over the keys. Software development works in a similar way.

1 SDLC analogy: Building a house



Remember this sequence: Understand > Plan > Build > Check > Release

1 Requirements: What does the customer want?

House: The customer explains the number of rooms, budget, kitchen size, parking, and other needs.

Software: The customer explains what features an application must have.

2 Design: How will we build it?

House: The architect creates the floor plan and decides where each room, door, and window will be.

Software: Developers and architects plan the screens, database, and system structure.

3 Coding: Build the actual product

House: Construction workers follow the approved plan and build the house.

Software: Developers write Java, Python, JavaScript, or other code to implement the features.

4 Testing: Check whether everything works

House: The inspector checks the doors, plumbing, electricity, and overall safety.

Software: Testers check login, payments, forms, and other features to find defects.

5 Deployment: Make it available to the customer

House: The completed house is handed over to the customer.

Software: The team releases the application to a production server so real users can use it.

2 Real-time example: Building a login

Imagine a customer asks a software company:

> “I want users to log in to my application using their email or phone number and password.”

How does the company turn this requirement into a working feature?

1

Phase 1: Requirements

People: Customer, Business Analyst, Product Owner

The customer explains what the login feature must do.

- Users can enter an email or phone number.
- Users can enter their password.
- Correct credentials allow login.
- Incorrect credentials display an error.
- Users can reset a forgotten password.

Output: A clear list of requirements or user stories.

2

Phase 2: Design

People: UI/UX Designer, Software Architect, Developers

The team decides how the login feature will work.

- Design the login screen.
- Decide how the frontend communicates with the backend.
- Design the database structure for user accounts.
- Plan secure password verification and session handling.

Output: Screen designs, technical architecture, and database/API designs.

3

Phase 3: Coding

People: Frontend and Backend Developers

Developers implement the planned feature.

- Create the email and password fields.
- Add the Login button.
- Write backend login logic.
- Verify credentials securely.
- Return success or an appropriate error.

Output: Working code that can be built and tested.

4

Phase 4: Testing

People: QA Testers and Developers

The tester checks whether the feature meets the requirements.

Test scenario	Expected result
Correct email and password	Login succeeds
Incorrect password	Error is displayed
Empty email field	Validation message appears
Invalid email format	Appropriate validation appears
Forgotten password	Reset process works

Output: Test results and bug reports. If a bug is found, the developer fixes it and the tester tests it again.

5

Phase 5: Deployment

People: DevOps Engineers, Developers, Release Team

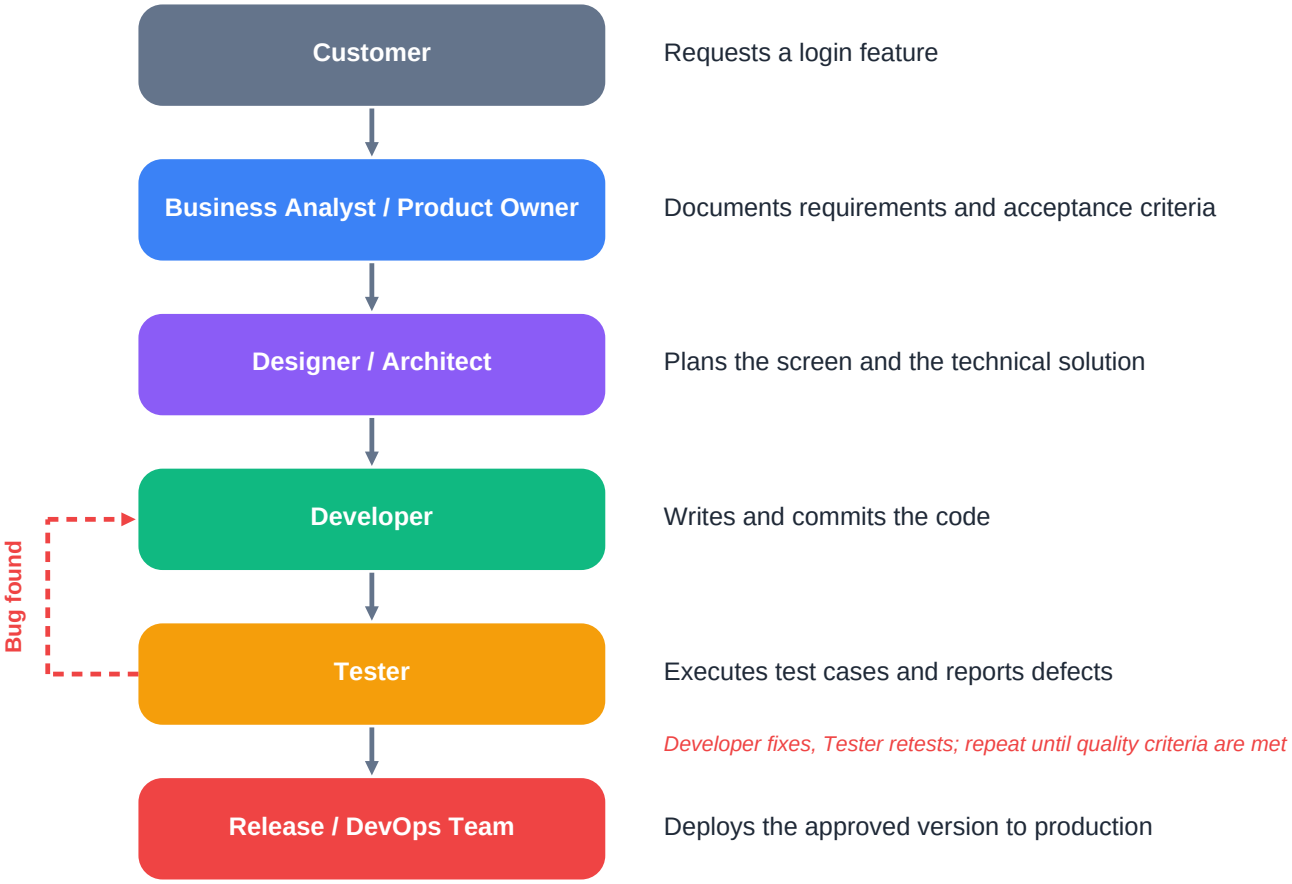
After the required approvals and checks, the team releases the login feature.

- Deploy the application to the production environment.
- Configure production settings and database connections.
- Verify that the login feature works in production.
- Monitor errors and application performance.

Output: The feature is available to real users.

3 How work flows between people

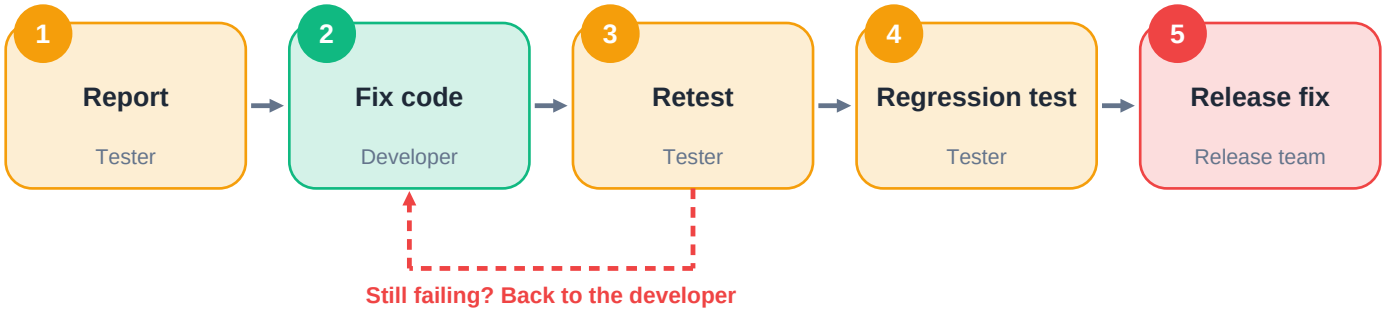
Let’s follow one requirement from the beginning to the end.



Note: In real companies, people can share responsibilities, and some phases overlap. Deployment is not the end of the entire software lifecycle; monitoring, maintenance, and further improvements continue afterward.

4 What happens when the tester finds a bug?

Suppose the tester enters a wrong password, but the application logs the user in anyway.



The bug-fix steps in detail

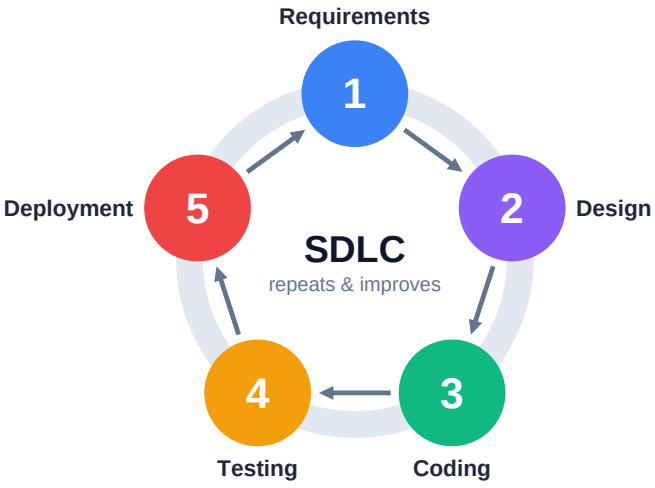
Step	Who	What happens
1	Tester	Reproduces the issue and records the steps, expected result, and actual result in Jira.
2	Developer	Investigates the code and fixes the password-validation logic.
3	Tester	Runs the failed test again to verify the fix.
4	Tester	Performs regression testing to ensure the change did not break other login features.
5	Release team	Includes the approved fix in a production release.

Why this matters: Testing is not simply the final step after coding. Testing and development often repeat until the feature meets its requirements.

5 Quick revision

Phase	Main question	Typical deliverable
Requirements	What should we build?	Requirements and user stories
Design	How should we build it?	Designs and technical plan
Coding	Can we implement it?	Source code
Testing	Does it work correctly?	Test results and defect reports
Deployment	Can real users use it?	Released production application

For a software tester, remember: understand the requirements before writing test cases, test the developer's implementation against those requirements, report defects clearly, retest fixes, and verify that the released feature works as expected.



The cycle never really ends

Deployment is not the finish line. After release, the team monitors the application, fixes problems, and plans improvements, so the next round of requirements, design, coding, testing, and deployment begins.

Testing is not a single step at the end either: it happens alongside development, and bugs loop back to the developer until the quality criteria are met.